MATHIIT.LIVE

Critical Security & Data Integrity Bug Report

Updated report — database exposure, account/session mix-up, worksheet integrity and authorization

Overall severity	**CRITICAL**
Highest-risk findings	Entire student database exposure; possible cross-student account/session mix-up
Evidence	Direct request formats and observed behavior supplied by the reporter; identifiers redacted
Important	No real credentials, student IDs, names, phone numbers or database records are reproduced here.

Security boundary
Student Browser → **mathiit.live** (CAPTCHA + authentication + authorization + validation + trusted timestamps/grading) → Supabase. The browser should not be a trusted database client.

Executive Summary

The worksheet platform exposes a Supabase API credential to the browser and permits direct backend interaction using that credential. The most serious issue is the apparent absence of a reliable server-side authorization boundary between a student's browser and student/worksheet records.

The updated evidence adds two particularly serious concerns: the reporter was able to retrieve the entire student database through an accessible backend resource, and a possible access-token collision/session mix-up caused the browser to display a friend's account after a normal submission.

The report also records the incorrect-mark calculation, repeated submission, client-controlled timing/status, and cross-student worksheet manipulation concerns.

Priority findings

#	Finding	Severity	Status
01	Entire student database exposure	CRITICAL	REPORTED / OBSERVED
02	Access-token collision causing account/session mix-up	CRITICAL	OBSERVED
03	Cross-student worksheet/attempt manipulation	CRITICAL	REQUEST PATH DEMONSTRATED
04	Direct Supabase API access	CRITICAL	CONFIRMED
05	Client-controlled status/timing/re-submission	CRITICAL	DEMONSTRATED
06	Incorrect worksheet marks	HIGH	OBSERVED

01 Entire Student Database Exposure

CRITICAL

Severity: Critical

The reporter states that a backend resource accessible with the browser-provided Supabase credential returned student records at a database-wide scope. The attempted query was intended to inspect the available student data and was able to return the full student dataset.

The evidence included a Supabase client initialized with the browser-provided credential and a table query. The specific table query that was confirmed to return data was **teacher_centers**; the reporter also states that an accessible public-summary/RPC path exposed student information. The exact successful student-database response is not reproduced here.

Request structure used to access the database

```
from supabase import create_client

url = "https://sthomkdkjwxwbolomovx.supabase.co"
key = "<REDACTED_BROWSER_API_KEY>"

supabase = create_client(url, key)

# Direct database query using the browser-exposed credential
response = supabase.table("<REDACTED_TABLE>").select("*").execute()

print(response.data)
```

The reported student records contained fields including student name, student ID/code, class, status, parent/contact information, previous academic marks and student status information.

Impact
This is the highest-priority privacy issue. A normal student should never be able to enumerate or download another student's personal or academic records. Database-level access must be constrained by server-side authorization and Supabase RLS.

02 Access-Token Collision / Account Mix-Up

CRITICAL

Severity: Critical

The reporter observed a serious account-isolation problem involving worksheet access tokens. After submitting a worksheet normally in the browser, the application unexpectedly displayed a friend's name and marks. The reporter states that this happened by coincidence because the affected account belonged to a friend.

The reported behavior suggests that worksheet access tokens may not be guaranteed to be unique, or that token-to-student/session resolution is not being validated correctly. A token collision or incorrect token lookup could associate one student's worksheet session with another student's account.

Reported sequence

```
1. Student opens/submits their own worksheet normally.
2. Worksheet submission completes.
3. Browser unexpectedly displays another student's name/marks.
4. The other student happened to be a friend, allowing the account mix-up to be noticed.
5. The reporter then observed that worksheet attempt state could be sent with
the friend's student_id and worksheet_id.
6. The worksheet was later submitted and another request was sent with the
friend's identifiers and status = "submitted".
```

Representative status payload structure

```
{
"worksheet_id": "<FRIEND_WORKSHEET_ID>",
"student_id": "<FRIEND_STUDENT_ID>",
"access_token": "<REDACTED_ACCESS_TOKEN>",
"status": "in_progress",
"started_at": "<TIMESTAMP>"
}
```

Later submission structure

```
{
"worksheet_id": "<FRIEND_WORKSHEET_ID>",
"student_id": "<FRIEND_STUDENT_ID>",
"access_token": "<REDACTED_ACCESS_TOKEN>",
"status": "submitted"
}
```

Impact

If the server accepts a client-provided student_id together with a token without independently verifying ownership, one student's session could become associated with another student's account. This can expose marks and enable cross-student record manipulation.

Required fix

Generate cryptographically random, sufficiently long access tokens; enforce uniqueness at the database level; bind each token to exactly one worksheet attempt and authenticated student; reject mismatched student/token/attempt combinations; and never use token equality alone as proof of account ownership.

03 Cross-Student Worksheet / Attempt Manipulation

CRITICAL

Severity: Critical

The worksheet attempt API accepts identifiers such as student_id, worksheet_id and attempt_id from the client. The supplied evidence demonstrates direct requests that create or target worksheet attempts. The security issue is that these identifiers must not be trusted as proof that the requester owns the referenced record.

Request structure

```
PATCH /rest/v1/worksheet_attempts?id=eq.<REDACTED_ATTEMPT_ID>

{
"student_id": "<REDACTED_STUDENT_ID>",
"worksheet_id": "<REDACTED_WORKSHEET_ID>",
"status": "submitted",
"time_spent_seconds": "247"
}
```

Grading request structure

```
POST /rest/v1/rpc/grade-attempt

{
"attempt_id": "<REDACTED_ATTEMPT_ID>"
}
```

Potential impact

If ownership checks are missing, an attacker who knows or obtains another student's identifiers may be able to interfere with that student's worksheet attempt, including state, submission, timing or grading-related information.

Expected behavior
Every read/write must derive the authenticated student on the server and verify ownership of the target attempt. A client-supplied student_id must never establish authorization.

04 Direct Supabase API Access

CRITICAL

Severity: Critical

The browser exposes the Supabase project URL and API credential. The reporter reproduced a direct request to the Supabase PostgREST root using those values.

```
import requests

url = "https://sthomkdkjwxwbolomovx.supabase.co/rest/v1"
key = "<REDACTED_API_KEY>"

headers = {
"apikey": key,
"Authorization": "Bearer " + key
}

response = requests.get(url, headers=headers)

print(response.status_code)
print(response.text)
```

This demonstrates why the database cannot rely on the secrecy of a browser-side credential. A public/anonymous key can be visible by design, but its permissions must be tightly restricted by RLS and backend authorization.

05 Client-Controlled Status, Timing & Re-submission

CRITICAL

Severity: Critical

The worksheet workflow allows request bodies to contain attempt status and timing information. The reporter also states that the same worksheet can be submitted again with different values.

Examples of client-supplied fields

```
{
"status": "in_progress"
}

{
"status": "submitted",
"time_spent_seconds": "247"
}
```

Integrity risks

• A student may be able to alter the recorded completion time.
• A student may be able to replay a worksheet submission.
• A student may be able to change an attempt's workflow state.
• If grading trusts client-controlled data, marks may be affected.

Low-bandwidth design

If the reason for frequent Supabase writes is low-internet consumption, the architecture can still be improved: send worksheet events to mathiit.live, where they are validated and timestamped. Events can be buffered and sent periodically or as a final batch rather than exposing the database directly to the student browser.

06 Incorrect Worksheet Marks

CRITICAL

Severity: High

A separate grading defect was observed: when one question is answered incorrectly on a 15-question worksheet, the resulting score can be displayed as 13/15 rather than the expected score based on the actual number of correct answers.

This is primarily a correctness/data-integrity defect rather than the core authorization vulnerability. It should nevertheless be fixed because worksheet scores are academic records.

> **Expected behavior**
> The score must be calculated from the authoritative answer key and the student's submitted answers on the server. The browser should display the server-calculated result rather than calculate or submit the final mark.

Recommended Remediation Architecture

The recommended architecture is to place mathiit.live between the student browser and Supabase. CAPTCHA should be applied specifically at the **Student Browser** → **mathiit.live** boundary.

```
Student Browser
|
| HTTPS
| CAPTCHA
v
mathiit.live
|
| Authenticate student
| Verify worksheet ownership
| Validate access token
| Validate state transitions
| Generate server-side timestamps
| Calculate grades server-side
v
Supabase
|
| RLS + service-side authorization
v
Trusted worksheet data
```

Recommended controls

1. **Unique access tokens:** use cryptographically secure random tokens, enforce a UNIQUE database constraint, and bind each token to exactly one attempt/student/worksheet.

2. **Server-side identity:** never trust student_id supplied by the browser. Resolve the student from the authenticated session.

3. **Ownership checks:** verify student → attempt → worksheet relationships on every read/write.

4. **Server-side timestamps:** record authoritative event times at mathiit.live. Browser timestamps should be treated as untrusted.

5. **Server-side grading:** calculate marks from trusted answers and answer keys; never accept a final score from the client.

6. **Supabase RLS:** restrict direct access so students cannot enumerate or modify unrelated rows.

7. **CAPTCHA:** place CAPTCHA between the student browser and mathiit.live to reduce automated abuse. CAPTCHA is an anti-abuse control, not an authorization mechanism.

8. **Audit logging:** log account, attempt, token, state-transition and grading events so cross-student anomalies can be detected.

9. **Credential rotation:** the API credential included in the original evidence should be rotated if it is a live credential.

Verification & Evidence Matrix

Finding	Evidence status
Finding	Evidence status
Direct PostgREST/API request	CONFIRMED WORKING
Direct worksheet_attempts interaction	CONFIRMED WORKING
Worksheet attempt creation payload	CONFIRMED WORKING
Direct attempt modification structure	DEMONSTRATED — authorization must be verified
Grading RPC request structure	DEMONSTRATED — authorization must be verified
Entire student database exposure	REPORTED / OBSERVED BY TESTER — preserve server response
Access-token/account mix-up	OBSERVED — browser displayed friend's account/marks
Incorrect marks (e.g. 13/15 for one wrong answer)	OBSERVED
teacher_centers table query	CONFIRMED BY REPORTER
Some other RPC/table tests	NOT CONFIRMED

Evidence handling
Keep the original server responses, screenshots and logs privately for the vendor. Do not distribute real API keys, access tokens, student UUIDs, names, contact details or downloaded student records. The examples in this report intentionally use placeholders.